Complete required programs for the corrected quotient census

The three GAP files reconstruct the marked group, evaluate all thirty literal source words, and exhaust every subgroup conjugacy class. The Python file reconciles the complete historical containment tables with the fresh census. Each file is included in full. The separate raw data file `source_tables.json` contains all 145 printed rows; it is required data, not omitted code. The portable `check.py` orchestrates these programs, checks bindings, replays eight witnesses and tests corruptions. Its complete file is separately exported. No target list of semisparse groups is an enumeration input.

programs/ambient_marking.g

```
# Ambient marking: seed-search algorithm reused from accepted phase40, with reversed order.
# No subgroup or geometry census is an input.
if LoadPackage("tomlib")<>true then Error("TomLib is required");fi;
SizeScreen([100000,24]);;
CheckL084:=function(test,msg) if not test then Error(msg);fi;end;;
G:=UnderlyingGroup(TableOfMarks("J1"));;
CheckL084(Size(G)=175560,"ambient J1 order");;
FindJankoSeed:=function(K)
 local a,b,c,d,iv,bs;
 a:=GeneratorsOfGroup(K)[1];
 CheckL084(Order(a)=2,"ambient first generator");
 iv:=Filtered(Elements(Centralizer(K,a)),x->Order(x)=2);
 for d in iv do
  if d=a then continue;fi;
  bs:=Filtered(Elements(Centralizer(K,d)),x->Order(x)=2 and Order(a*x)=5);
  for c in iv do
   if Order(c*d)<>5 then continue;fi;
   for b in bs do
    if Order(b*c)=3 and (a*b*c)^5=One(K)
      and ((d*c*b)^5*a)^3=One(K) and Group(a,b,c,d)=K then
     return [a,b,c,d];
    fi;
   od;
  od;
 od;
 Error("no source seed");
end;;
s:=FindJankoSeed(G);;
jmarks:=Reversed(s);;
pmarks:=[(1,2)(3,6)(4,7)(5,10)(8,14)(9,15)(11,19)(12,16)(13,17)(18,20),
(1,3)(2,5)(4,8)(6,13)(7,12)(9,17)(10,15)(11,18)(14,19)(16,20),
(1,3)(2,6)(4,9)(5,11)(7,15)(8,16)(10,19)(12,14)(13,18)(17,20),
(1,4)(2,7)(3,8)(5,12)(6,14)(9,18)(10,16)(11,17)(13,19)(15,20)];;
P:=Group(pmarks);; W:=DirectProduct(G,P);; ej:=Embedding(W,1);; ep:=Embedding(W,2);;
marks:=List([1..4],i->Image(ej,jmarks[i])*Image(ep,pmarks[i]));;
CheckL084(Group(marks)=W and Size(W)=600415200,"full product generation");;
H0:=Group(marks{[2,3,4]});;H3:=Group(marks{[1,2,3]});;om:=(marks[1]*marks[2]*marks[3])^5;;
CheckL084(ForAll(marks,x->Order(x)=2),"four involutions");;
CheckL084(List([1..3],i->Order(marks[i]*marks[i+1]))=[5,3,5],"exact adjacent labels");;
CheckL084(ForAll([[1,3],[1,4],[2,4]],ij->Comm(marks[ij[1]],marks[ij[2]])=One(W)),"all string relations");;
CheckL084((marks[2]*marks[3]*marks[4])^5=One(W),"hemi-icosahedral presentation relator");;
CheckL084(Size(P)=3420 and Size(H0)=60 and Size(H3)=120 and Order(om)=2 and
ForAll(GeneratorsOfGroup(H3),x->Comm(x,om)=One(W)),"exact local groups and central antipode");;
Print("MODEL ",Size(W)," ENDGROUPS ",Size(H0)," ",Size(H3)," OMEGA ",Order(om),"\n");
```

programs/source_maximals.g

```
# Literal L095 arXiv v1, Table 1 (physical page 11); source row 58 is recorded literally.
v1:=(marks[1]*marks[2]*marks[3])^5*marks[4];;
v2:=v1*marks[3];;v3:=v2*marks[2];;v4:=v3*marks[1];;v5:=v4*marks[2];;v6:=v5*marks[1];;
sourceMax:=[
[1,[v1^2,(v2*v1)^2,v3^5,(v5*v2*v1)^2]],
[2,[(v2*v1^2)^2,(v6^2*v4*v1)^2]],
[3,[v1^2*v2*v4^2*v6*v3,v3^2*v1*v4*v5^2*v6*v1]],
[4,[v5*v1*v5*v4*v1*v2*v4*v1,v2^2*v1*v5*v3^2*v2*v4]],
[5,[v1*v3*v4^2*v5*v2^2*v1,v1*v6^2*v2*v3*v5*v2*v4]],
[6,[v1*v3*v6*v5^2*v3,v2^2*v6^2*v1*v6*v2]],
[7,[v3*v4*v1*v6*v5*v3*v1,v5*v6*v5*v1*v2*v5^2,v6*v2^2*v6*v4*v1*v4*v2]],
[8,[v5*v6^2*v3*v5*v1,v3^2*v2*v3*v1*v2*v3,v3*v5*v2*v6*v5*v3*v6]],
[9,[v5*v6^2*v3*v5*v1,v2*v3^3*v2*v3*v1]],
```

```
[11,[v2*v5*v2*v1*v3*v4*v3,v4^2*v1*v6*v5*v3*v2*v1^2,v1*v3*v4*v5*v6*v1*v4*v3*v1]],
[13,[v1*v3^6*v1^2,v5*v3*v5*v6*v4*v6^2*v2*v1]],
[16,[v4*v6^2*v2*v3*v4^2*v2,v1*v5*v6*v5^2*v6*v3*v4]],
[17,[v1*v2*v3*v6*v1*v4*v2*v1,v4*v6^2*v2*v3*v4^2*v2]],
[19,[v6*v3*v4*v6*v5^2,v6*v2^3*v6^2*v1^3,v5*v6^2*v5*v3*v5*v4*v2*v1]],
[20,[v3^2*v5^2*v6*v5*v4*v1,(v6*v1)^2*v2*v5*v4*v5*v1]],
[22,[v1*v5*v4*v2,v5*v1*v3*v5*v1*v4*v2^2*v6]],
[23,[v1^3,v4*v3*v4*v2*v6*v3*v2^2*v4,v4*v6*v5*v4^2*v3*v5*v3*v4]],
[25,[v3*v1^2*v5^2*v6*v4*v5,v5*v6^2*v3*v5*v4*v2*v5*v4]],
[29,[(v3*v1)^2*v5*v2*v1*v4,v6^2*v3*v4*v5^2*v4*v5]],
[30,[v3^6*v1^3,v6*v3^2*v4*v3^2*v1*v4*v1]],
[33,[v4*v5*v4*v3*v1^2,v2*v1^3*v2^4]],
[42,[v1*v5*v3*v1*v4*v6^2*v1^2,v3*v6*v2*v6*v4*v6^3*v1]],
[48,[v6*v4*v3*v4*v1*v5*v2*v1*v6*v1,v3*v6*v5*v2*v3*v4^2*v5*v6*v1]],
[54,[v3*v4*v1*v5*v2^2*v4,v1^2*(v2^2*v1)^2]],
[58,[v6*v1^2*v2*v5*v6*v3*v4*v1,v6*v1^2*v3*v5^2*v1^2*v4*v1]],
[64,[v5*v3*v2*v5*v6*v1*(v3*v1)^2,v2^2*v4*v1^2*v4*v3*v6*v3*v2]],
[70,[v2^3*v6^2*v1*v6,v5*v3*v5*v4*v2*v3*v4*v1*v4*v2,v1*v5*v1*v3*v5*v2*v1*v6*v5*v6]],
[81,[v5*v3*v1*v2*v4*v2*v1*v6*v1,(v4*v5*v4)^3]],
[95,[v1*v2*v1*v4*v5^2*v4*v6*v4,v5*v3*v5*v4*v5*v6*v4*v2*v6*v1^2]],
[127,[v1*v2*v3*v1*v2*v4*v1^2*v3^2*v1]]];;
```

programs/quotient_census.g

```
# Full required census; run from the packet root. GAP exact algorithms are retained.
Read("programs/ambient_marking.g");;
Read("programs/source_maximals.g");;
CensusL084:=function()
 local cls,cc,prod,ClassIndex,bad,omid,omcent,rows,good,rejected,goodEdges,
 i,n,ncls,ni,invol,hn,dn,j,k,idx,maxidx,anchors,h,row,target,children,
 targets,child,niBad,x,b,q,witnesses,genfile,reportfile,types,t,par;
 t:=Runtime();
 cls:=ConjugacyClasses(W);
 ClassIndex:=function(x)
  return PositionProperty(cls,c->Order(Representative(c))=Order(x) and x in c);
 end;
 prod:=Set(Concatenation(List(Elements(H0),u->List(Elements(H3),v->u*v))));
 CheckL084(Length(prod)=1200,"complete end product");
 bad:=Set(List(Difference(prod,[One(W),om]),ClassIndex));
 omid:=ClassIndex(om);omcent:=Size(Centralizer(W,om));
 CheckL084(Length(cls)=180 and Length(bad)=47 and not omid in bad,"element classes");
 cc:=ConjugacyClassesSubgroups(W);
 CheckL084(Length(cc)=1262,"complete subgroup-class census");
 Print("CLASSES ",Length(cc),"\n");
 rows:=[];good:=[];rejected:=[];goodEdges:=[];
 for i in [1..Length(cc)] do
  n:=Representative(cc[i]);ncls:=ConjugacyClasses(n);
  ni:=List(ncls,c->ClassIndex(Representative(c)));
  niBad:=Intersection(ni,bad);
  if niBad=[] then
   Add(good,i);
   invol:=Sum(Filtered([1..Length(ni)],j->ni[j]=omid),j->Size(ncls[j]));
   hn:=omcent*invol/(60*Size(n));dn:=Size(W)/(120*Size(n))-hn/2;
   CheckL084(IsInt(hn) and IsInt(dn) and hn>=0 and dn>=0,"integer facet counts");
   Add(rows,[i,Size(n),dn,hn]);
  else Add(rejected,[i,niBad[1]]);fi;
 od;
 CheckL084(Length(good)=137,"all semisparse classes");
 CheckL084(Number(rows,r->r[4]=0)=65,"all dodecahedral facets");
 CheckL084(Number(rows,r->r[3]=0)=1,"all hemi-dodecahedral facets");
 CheckL084(Number(rows,r->r[3]>0 and r[4]>0)=71,"mixed facets");
 for i in good do
  n:=Representative(cc[i]);
  for k in MaximalSubgroupClassReps(n) do
   idx:=PositionProperty(cc,c->Size(Representative(c))=Size(k) and k in c);
   CheckL084(idx in good,"downward closure");AddSet(goodEdges,[i,idx]);
  od;
 od;
 maxidx:=Difference(good,Set(List(goodEdges,e->e[2])));
 CheckL084(Length(maxidx)=26 and Length(goodEdges)=349,"complete maximal containment");
 anchors:=[];
 for row in sourceMax do
  h:=Group(row[2]);idx:=PositionProperty(cc,c->Size(Representative(c))=Size(h) and h in c);
  Add(anchors,[row[1],idx,Size(h)]);
 od;
 CheckL084(Set(List(Filtered(anchors,a->not a[2] in good),a->a[1]))=[13,70,95,127],"literal maximal deletions");
 CheckL084(Set(List(Filtered(anchors,a->a[2] in good),a->a[2]))=maxidx,"all surviving literal maxima");
 # These children are identified by literal source inclusion rows, group type,
 # and (for row 91) the printed normalizer quotient order.
 targets:=List([13,70,95,127],lab->[lab,Group(First(sourceMax,a->a[1]=lab)[2])]);
 h:=targets[1][2];children:=List(ConjugacyClassesSubgroups(h),Representative);
 for row in [[78,45],[111,15]] do
  target:=Filtered(children,k->Size(k)=row[2]);
  CheckL084(Length(target)=1 and IsCyclic(target[1]),"unique cyclic child of row13");
  Add(targets,[row[1],target[1]]);
 od;
 h:=targets[2][2];children:=List(ConjugacyClassesSubgroups(h),Representative);
 target:=Filtered(children,k->Size(k)=30 and IsCyclic(k));
 CheckL084(Length(target)=1,"unique cyclic row90");Add(targets,[90,target[1]]);
```

```
 target:=Filtered(children,k->Size(k)=30 and not IsAbelian(k) and Index(Normalizer(W,k),k)=12);
 CheckL084(Length(target)=1 and StructureDescription(target[1])="C3 x D10","row91 type and normalizer");
 Add(targets,[91,target[1]]);
 SortBy(targets,a->a[1]);witnesses:=[];
 for row in targets do
  h:=row[2];ncls:=ConjugacyClasses(h);
  x:=First(List(ncls,Representative),u->ClassIndex(u) in bad);
  CheckL084(x<>fail,"each deleted row fails conjugacy avoidance");
  b:=First(Difference(prod,[One(W),om]),u->IsConjugate(W,x,u));
  q:=RepresentativeAction(W,x,b);
  CheckL084(q<>fail and q in W and x in h and x^q=b and b in prod and not b in [One(W),om],"actual missed
  conjugate witness");
  idx:=PositionProperty(cc,c->Size(Representative(c))=Size(h) and h in c);
  Add(witnesses,[row[1],idx,Size(h),Order(x),ClassIndex(x)]);
  PrintTo(Concatenation("witness_",String(row[1]),".g"),
   "# Literal permutation witness for deleted source row ",row[1],"\n",
   "rejectedGenerators:=",GeneratorsOfGroup(h),";;\n",
   "violatingElement:=",x,";;\n","conjugator:=",q,";;\n","endProductElement:=",b,";;\n");
 od;
 CheckL084(List(witnesses,a->a[1])=[13,70,78,90,91,95,111,127],"all eight literal deletion labels");
 # Every subgroup representative is retained in the computational evidence.
 genfile:="computed_subgroups.g";PrintTo(genfile,"# Fresh complete GAP subgroup-class
 representatives.\nsubgroupGenerators:=[\n");
 for i in [1..Length(cc)] do
  AppendTo(genfile,GeneratorsOfGroup(Representative(cc[i])));
  if i<Length(cc) then AppendTo(genfile,",");fi;AppendTo(genfile,"\n");
 od;AppendTo(genfile,"];;\n");
 # Normalizer quotients and subgroup descriptions are computed for every survivor.
 types:=List(good,function(i)
  local n; n:=Representative(cc[i]);
  return [i,StructureDescription(n),StructureDescription(Normalizer(W,n)/n)];
 end);
 reportfile:="census.json";
 PrintTo(reportfile,"{\n\"ambient_order\":",Size(W),",\n\"subgroup_classes\":",Length(cc),
 ",\n\"element_classes\":",Length(cls),",\n\"bad_classes\":",bad,
 ",\n\"omega_class\":",omid,",\n\"omega_centralizer\":",omcent,
 ",\n\"survivors\":",rows,",\n\"rejected\":",rejected,
 ",\n\"maximal_indices\":",maxidx,",\n\"maximal_edges\":",goodEdges,
 ",\n\"source_maximal_anchors\":",anchors,",\n\"deletion_witnesses\":",witnesses,
 ",\n\"survivor_types\":",types,"\n}\n");
 Print("L084_CENSUS_PASS classes=1262 survivors=137 maximal=26 section_regular=66 same_local=65 mixed=71
 milliseconds=",Runtime()-t,"\n");
end;;
CensusL084();
QUIT;
```

accounting.py

```
#!/usr/bin/env python3
"""Exact historical-table matching of a fresh complete GAP quotient census."""
import copy,json,re,math
import networkx as nx
DELETED={13,70,78,90,91,95,111,127}
ORDER=600415200

def need(ok,why):
    if not ok:raise ValueError(why)

def descriptor_order(s):
    s=s.split('(~=')[0].strip()
    s=s.replace('PSL(2,19)','3420').replace('PSL(2,11)','660').replace('GL(2,4)','180').replace('L2(19)','3420')
    .replace('L2(11)','660').replace('J1','175560')
    s=re.sub(r'(?<![A-Za-z0-9])(192|22|23|32)(?![0-9])',lambda m:str({22:4,23:8,32:9,192:361}[int(m[1])]),s)
    s=re.sub(r'([CASD])(\d+)',lambda m:str(math.factorial(int(m[2]))//(2 if m[1]=='A'else 1))if m[1]in'AS'else
    m[2],s)
    ns=[int(x)for x in re.findall(r'\d+',s)]
    # Literal PDF extraction of the four displayed powers of primes.
    return math.prod(ns)

def canonical_direct(s):
    """Elementary direct-product identities for the printed automorphism columns.
    No semidirect-product association is inferred beyond the literal two
    Frobenius factors 7:3 and 19:3 occurring in those columns.
    """
    s=s.replace('PSL(2,19)','L').replace('L2(19)','L').replace('GL(2,4)','C3 x A5')
    s=s.replace('×','x').replace(' ','')
    for a,b in [('(C7:C3)','F21'),('(7:3)','F21'),('C7:C3','F21'),('7:3','F21'),('C19:C3','F57'),('19:3','F57')]
    :s=s.replace(a,b)
    result=[]
    def cyclic(n):
        for p in [2,3,5,7,11,19]:
            q=1
            while n%p==0:q*=p;n//=p
            if q>1:result.append('C'+str(q))
        need(n==1,'explicit cyclic factorization')
    for t in s.split('x'):
        if t in ['J1','L','A5','S3','F21','F57','D10']:result.append(t)
        elif t=='D12':result.extend(['C2','S3'])
```

```
        elif t=='D20':result.extend(['C2','D10'])
        elif t in ['22','23','32']:
            result.extend(['C'+t[0]]*int(t[1]))
        elif t.startswith('C') and t[1:].isdigit():cyclic(int(t[1:]))
        elif t.isdigit():cyclic(int(t))
        else:raise ValueError('unrecognized automorphism factor '+t)
    return sorted(result)

def facets(row):
    text=row['facets_literal_extracted']
    if row['label']==1:return 0,int(text)
    if row['section_regular_table']:return int(text),0
    vals=list(map(int,re.findall(r'\d+',text)));need(len(vals)==2,'two facet counts');return tuple(vals)

def reconcile(census,rows):
    need(census['ambient_order']==ORDER and census['subgroup_classes']==1262,'complete ambient census')
    need(census['element_classes']==180 and len(census['bad_classes'])==47,'complete forbidden classes')
    ss=census['survivors'];good={r[0]for r in ss}
    need(len(ss)==len(good)==137,'distinct survivors')
    no={r[0]for r in census['rejected']};need(not good&no and good|no==set(range(1,1263)),'every class tested')
    need(all(r[1]in census['bad_classes']for r in census['rejected']),'every rejection has forbidden class')
    need(len(rows)==145 and {r['label']for r in rows}==set(range(1,146)),'complete historical tables')
    need([r[0]for r in census['deletion_witnesses']]==sorted(DELETED),'exact eight source deletions')
    need(all(r[1]in no and r[4]in census['bad_classes']for r in census['deletion_witnesses']),'deleted rows
    fail')
    anchors={i:(63 if lab==58 else lab)for lab,i,size in census['source_maximal_anchors']if i in good}
    need(len(anchors)==26 and set(anchors)==set(census['maximal_indices']),'all literal surviving maxima')
    S=nx.DiGraph();T=nx.DiGraph();source_by_id={r['label']:r for r in rows}
    for r in rows:
        lab=r['label']
        if lab in DELETED:continue
        d,h=facets(r);den=120*d+60*h;need(ORDER%den==0,'integral printed group order')
        size=ORDER//den
        if lab not in [21,27]:need(descriptor_order(r['group_literal_extracted'])==size,'printed subgroup order
        '+str(lab))
        else:need(size=={21:456,27:342}[lab],'explicit arxiv group-column correction')
        S.add_node(lab,color=(size,d,h,lab if lab in anchors.values()else 0))
    for r in rows:
        if r['label']not in DELETED:
            need(not DELETED.intersection(r['subgroups']),'no deleted child of a surviving parent')
            S.add_edges_from((r['label'],i)for i in r['subgroups'])
    for i,o,d,h in ss:
        need(type(d)is int and type(h)is int and min(d,h)>=0,'nonnegative integral facets')
        need(o*(120*d+60*h)==ORDER,'full facet partition')
        T.add_node(i,color=(o,d,h,anchors.get(i,0)))
    edges=set(map(tuple,census['maximal_edges']));need(len(edges)==349,'complete distinct containment edges')
    need(all(a in good and b in good for a,b in edges),'closure of all maximal subgroups')
    T.add_edges_from(edges)
    need(set(census['maximal_indices'])=={i for i in T if T.in_degree(i)==0},'all maximal classes')
    matcher=nx.algorithms.isomorphism.DiGraphMatcher(S,T,node_match=lambda x,y:x['color']==y['color'])
    need(matcher.is_isomorphic(),'complete historical and computed directed graphs')
    mapping=matcher.mapping
    need(len(mapping)==137 and set(mapping.values())==good,'full bijection of surviving labels')
    types={i:(g,q)for i,g,q in census['survivor_types']}
    need(set(types)==good,'every survivor has a subgroup and normalizer description')
    for lab,i in mapping.items():
        printed=source_by_id[lab]['automorphism_literal_extracted']
        need(canonical_direct(printed)==canonical_direct(types[i][1]),'actual normalizer quotient agrees with
        printed row '+str(lab))
    need(sum(r[3]==0 for r in ss)==65 and sum(r[2]==0 for r in ss)==1 and sum(r[2]>0 and r[3]>0 for r in
    ss)==71,'all corrected local types')
    result={'status':'PASS','source_rows':145,'surviving_rows':137,'maximal_rows':26,'section_regular':66,'same_
    local_type':65,'mixed':71,'directed_edges':349,'historical_to_computed':{str(k):v for k,v in
    sorted(mapping.items())},'deleted_labels':sorted(DELETED),'edition_crosswalk':{'arxiv_table1_label':58,'full
    _tables_label':63,'order':70,'journal_label_not_asserted':True,'group_column_corrections':{'21':'2^3 x
    (19:3), order456','27':'2 x (19:9), order342'}},'normalizer_quotients_matched':137}
    return result

def controls(census,rows):
    failures=[]
    def reject(name,c,r):
        try:reconcile(c,r)
        except (ValueError,KeyError,ZeroDivisionError):failures.append(name)
        else:raise ValueError('control passed '+name)
    c=copy.deepcopy(census);c['bad_classes'].pop();reject('omit_forbidden_element_class',c,rows)
    c=copy.deepcopy(census);c['survivors'].pop();reject('omit_one_subgroup',c,rows)
    c=copy.deepcopy(census);c['rejected'].pop();reject('skip_one_rejected_class',c,rows)
    c=copy.deepcopy(census);c['maximal_edges'].pop();reject('omit_containment_edge',c,rows)
    c=copy.deepcopy(census);c['maximal_indices'].pop();reject('wrong_maximal_count',c,rows)
    c=copy.deepcopy(census);c['deletion_witnesses'][0][0]=14;reject('alter_historical_deletion_label',c,rows)
    c=copy.deepcopy(census);c['survivors'][0][3]+=1;reject('change_local_facet_type',c,rows)
    c=copy.deepcopy(census);c['ambient_order']//=2;reject('omit_ambient_factor',c,rows)
    r=copy.deepcopy(rows);next(x for x in r if
    x['label']==1)['subgroups'].pop();reject('alter_printed_subgroup_link',census,r)
    r=copy.deepcopy(rows);next(x for x in r if
    x['label']==145)['facets_literal_extracted']='5003459';reject('alter_printed_facet_count',census,r)
    return failures
```